

基于深度视觉的自动驾驶汽车的3D点云地图构建存储与读取

一、任务描述

现在很多智能导航场景都涉及到激光(毫米波, 固态等)雷达和相机视觉信息融合, 这里激光雷达一般都是指多线激光雷达。但多线激光雷达动不动数万的价格, 让很多技术人员无法尝试。这里我使用一款深度相机来做3D点云和相机视觉的融合, 构建一个彩色3D点云场景。

二、任务目标

- 理解相机标定的原理和作用。
- 理解octomap的算法原理。
- 掌握rosviz的基本操作。
- 能够使用自动驾驶汽车进行3D建图和导航。

三、任务环境

- 操作系统: Ubuntu20.04 ROS-neotic
- 工具软件: NoMachine
- 必备硬件: 自动驾驶汽车

四、任务原理

4.1 相机标定

在图像测量过程以及机器视觉应用中, 为确定空间物体表面某点的三维几何位置与其在图像中对应点之间的相互关系, 必须建立相机成像的几何模型, 这些几何模型参数就是相机参数。在大多数条件下这些参数必须通过实验与计算才能得到, 这个求解参数的过程就称之为相机标定(或摄像机标定)。无论是在图像测量或者机器视觉应用中, 相机参数的标定都是非常关键的环节, 其标定结果的精度及算法的稳定性直接影响相机工作产生结果的准确性。因此, 做好相机标定是做好后续工作的前提, 提高标定精度是科研工作的重点所在。

任何理论物理模型都是在特定假设上对真实事物的近似, 然而在实际应用中存在误差, 普通相机的成像模型也不例外(透视投影)。实际中, 普通相机成像误差的主要来源有两部分, 第一是sensor制造产生的误差, 比如sensor成像单元不是正方形, sensor歪斜; 第二是镜头制造和安装产生的误差, 镜头一般存在非线性的径向畸变; 镜头与相机sensor安装不平行, 还会产生切向畸变。

小孔成像模型虽然充分考虑了相机内部参数对成像的影响, 但没有考虑成像系统另一个重要的部分, 镜头。镜头常用的有普通镜头、广角镜头、鱼镜头等, 在无人驾驶和视觉slam领域, 鱼镜头和广角镜头用的很多, 主要是视角很大, 可以观测到更多的信息。任何镜头都存在不同程度的畸变, 不同类型的镜头用到的畸变模型也不相同。根据镜头制造和成像的物理特性, 普通镜头主要考虑径向畸变和切向畸变, 且畸变模型都可以用多项式来近似。而对于大广角、鱼镜头, 普通镜头的物理模型不能适用了。

4.1.1 径向畸变

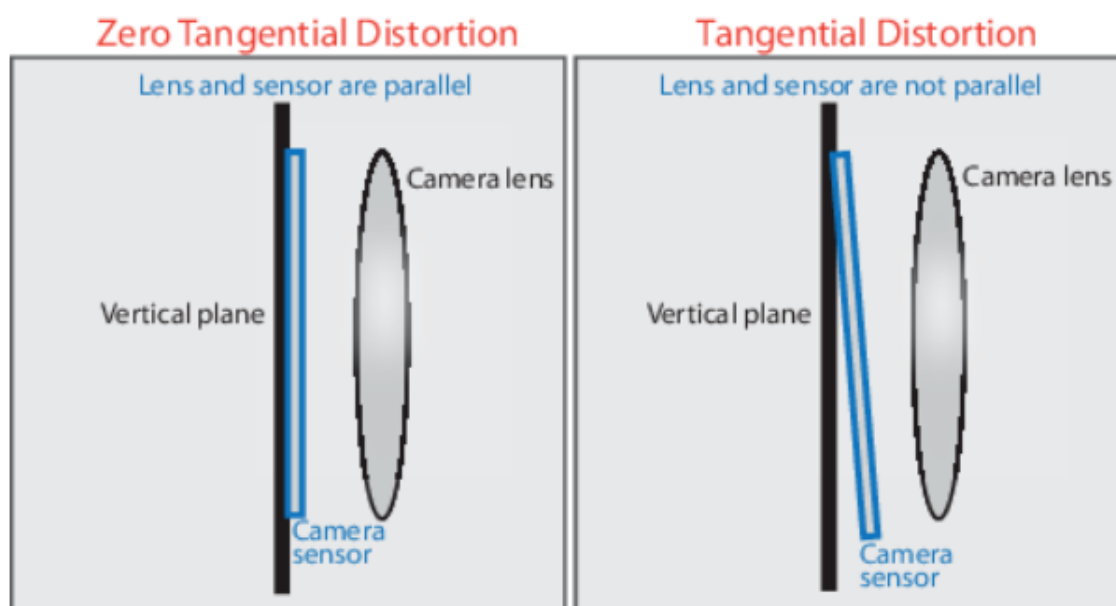
透过镜头边缘的光线很容易产生径向畸变，光线离镜头中心越远，畸变越大：



上图从左到右依次为无畸变、桶形畸变、枕形畸变。从图像可以看出，径向畸变以某一个中心往外延伸，且越往外，畸变越大；显然畸变与距离成一种非线性的变换关系。

4.1.2 切向畸变

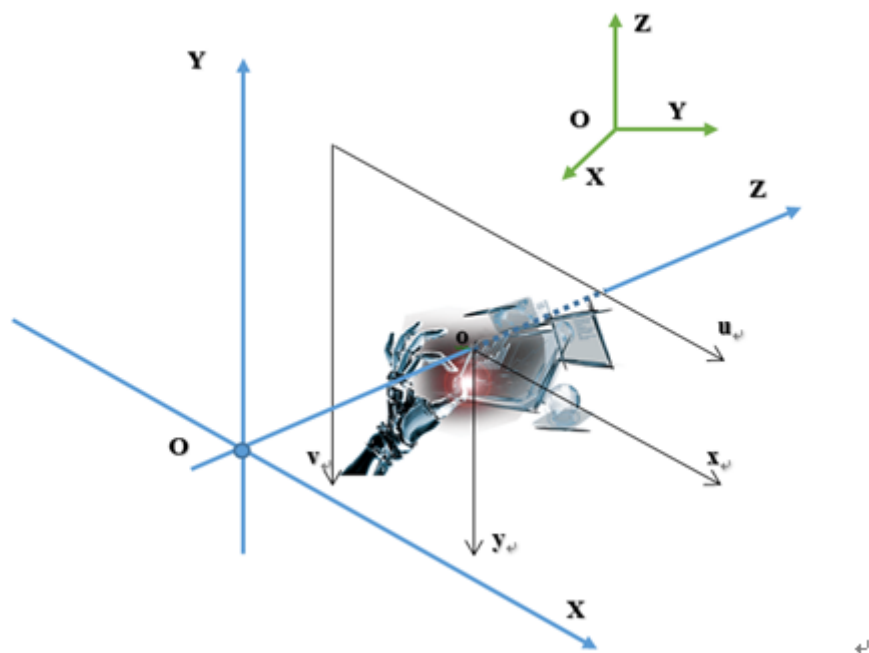
切向畸变主要发生在相机sensor和镜头不平行的情况下；因为有夹角，所以光透过镜头传到图像传感器上时，成像位置发生了变化。



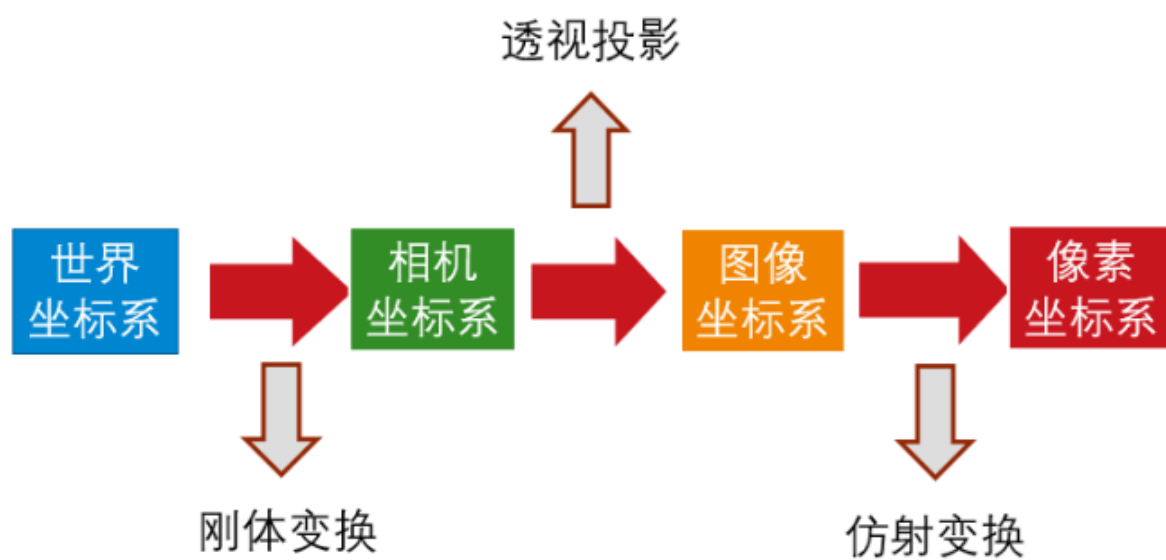
4.1.3 相机标定涉及到的坐标系

1. 世界坐标系：为了更好的描述相机在物理世界中的位置，所以世界坐标系定义不固定，根据不同的要求进行世界坐标系的设定，符号记为 $O_w - X_w Y_w Z_w$ ，单位为m。
2. 相机坐标系：以光心为原点，以光轴为Z轴建立的直角坐标系，符号记为 $O_c - X_c Y_c Z_c$ ，单位为m。
3. 图像坐标系：以光心，即图像中点为原点的二维坐标系，符号记为 $o - x y$ ，单位为mm。
4. 像素坐标系：以图像左上角为原点，水平向右为u轴，竖直向下为v轴，单位为pixel。

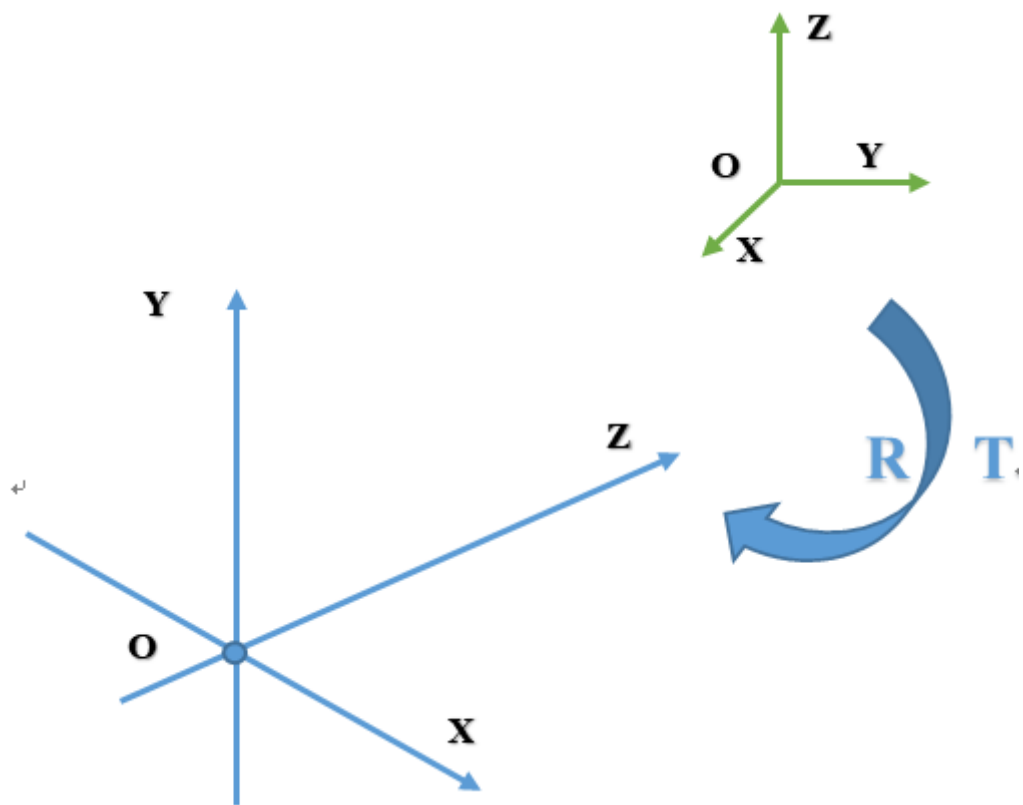
坐标系定义图如下：



到此，我们知道了四个坐标系的定义，现在我们就要寻找世界坐标的点与其对应像素坐标系点之间的转换关系。

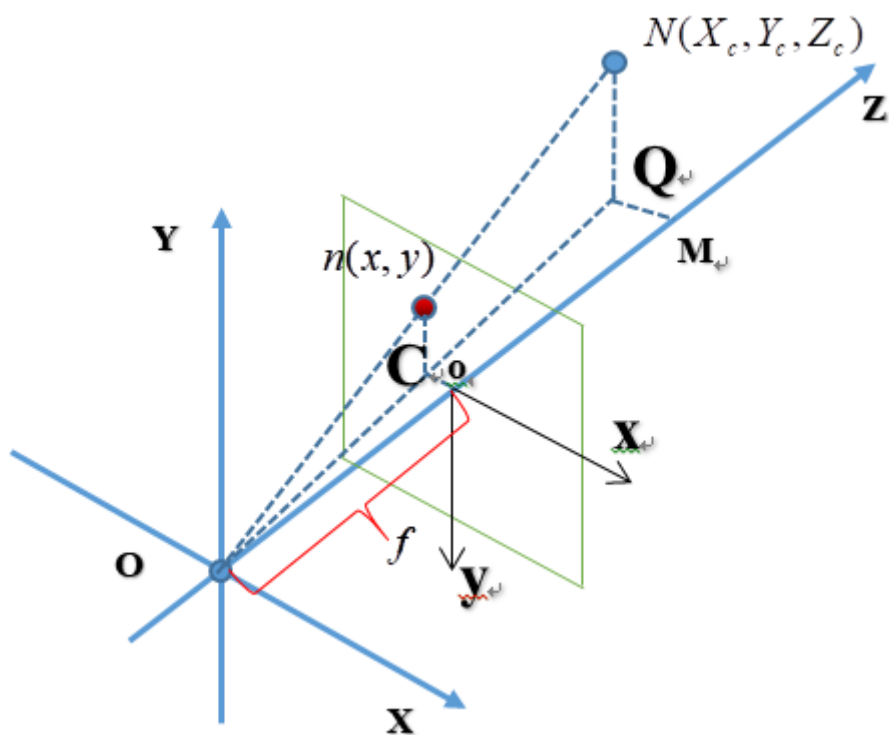


4.1.4 世界坐标系到相机坐标系



世界坐标系与相机坐标系之间的转换关系，从世界坐标系变换到相机坐标系属于刚体变换，所以只涉及到旋转与平移，即可用一个旋转矩阵 R 与一个平移向量 T 表示即可。而这个 R 与 T 被称为相机的外参矩阵。

4.1.5 相机坐标系到图像坐标系



上图中 N 点为相对于相机坐标系 $O_c - X_c Y_c Z_c$ 的点，与之图像对应的点为 n ，由三角形相似关系，即三角形 $MOCQ$ 与三角形 $oOcC$ 相似，三角形 $OcQN$ 与三角形 $Ocon$ 相似，这样：

$$\frac{MQ}{oC} = \frac{MO_c}{oO_c} = \frac{NQ}{nC} = \frac{X_c}{x} = \frac{Z_c}{f} = \frac{Y_c}{y}$$

整理可得：

$$x = f \frac{X_c}{Z_c}$$

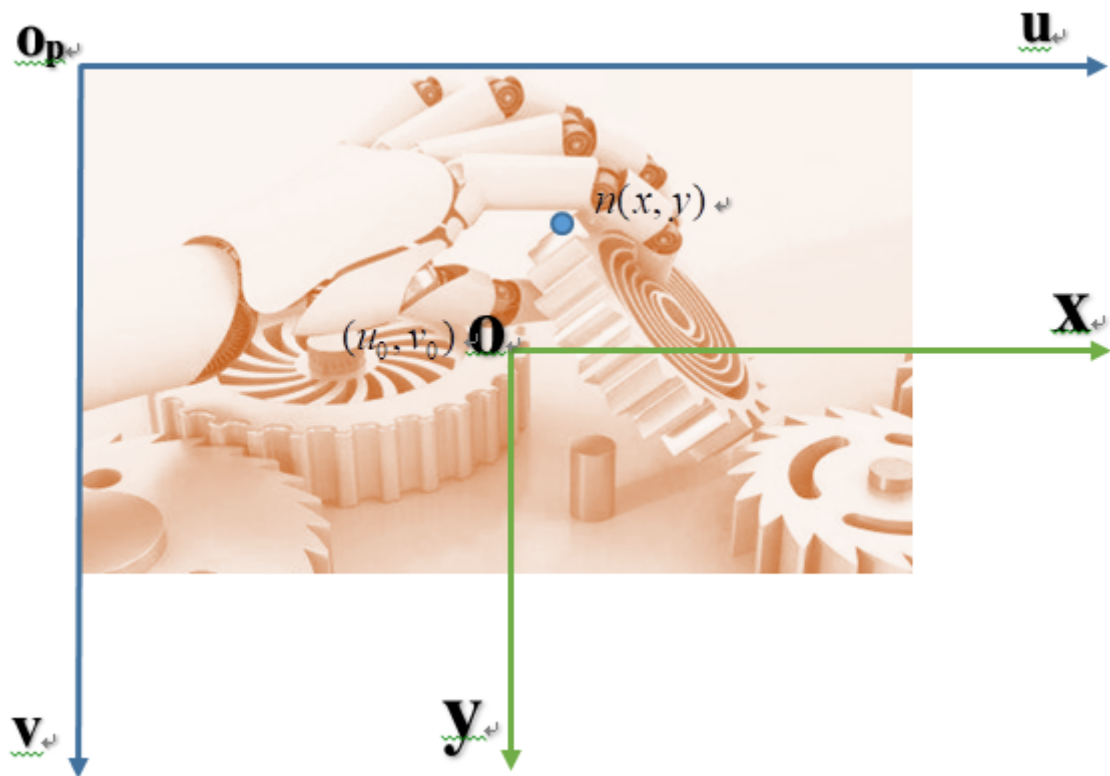
$$y = f \frac{Y_c}{Z_c}$$

整理成矩阵方程的形式，如下：

$$Z_c \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix}$$

上面只是得到了相机坐标系到图像坐标系之间的转换关系，其中n点的单位为mm，并不是pixel，下一步，我们还需要将其转换到像素坐标系上。

4.1.6 图像坐标系到像素坐标系



像素坐标系和图像坐标系都在成像平面上，只是各自的原点和度量单位不一样。图像坐标系的原点为相机光轴与成像平面的交点，通常情况下是成像平面的中点。图像坐标系的单位是mm，属于物理单位，而像素坐标系的单位是pixel，我们平常描述一个像素点都是几行几列。所以这二者之间的转换如下：其中 α_x 和 α_y 表示每一列和每一行分别代表多少mm，即 $1\text{pixel} = d_x\text{mm}$ 。

现在我们知道了n点在图像坐标系下的坐标为 (x, y) ，现在要得到n点在像素坐标系下的坐标，转换公式如下：

$$u = \frac{x}{dx} + u_0$$

$$v = \frac{y}{dy} + v_0$$

矩阵方程形式如下：

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{dx} & 0 & u_0 \\ 0 & \frac{1}{dy} & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

到此我们得到了世界坐标系与相机坐标系之间的转换关系、相机坐标系与图像坐标系之间的转换关系和图像坐标系与像素坐标系之间的关系，这样我们将这些变换连乘，就得到了世界坐标系到像素坐标系之间的变换关系，公式如下：

$$Z_c \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{dx} & 0 & u_0 \\ 0 & \frac{1}{dy} & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} R & T \\ 0 & 1 \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & 0 & u_0 & 0 \\ 0 & f_y & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{\text{内参矩阵}} \underbrace{\begin{bmatrix} R & T \\ 0 & 1 \end{bmatrix}}_{\text{外参矩阵}} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

4.2 octomap点云地图构建

4.2.1 什么是octomap

octomap是一种基于八叉树的三维地图创建工具，可以显示包含无障碍区域及未映射区域的完整3D图形，而且基于占有率栅格的传感器数据可以在多次测量中实现融合和更新；地图可提供多种分辨率，数据可压缩，存储紧凑。事实上，octomap的代码主要包含两个模块：三维地图创建工具octomap和可视化工具octovis。

由于八叉树的原因，它的地图像是很多个小方块组成的（很像《我的世界》这种沙盒类游戏）。当分辨率较高时，方块很小；分辨率较低时，方块很大。每个方块表示该格被占据的概率。因此你可以查询某个方块或点“是否可以通过”，从而实现不同层次的导航。简而言之，环境较大时采用较低分辨率，而较精细的导航可采用较高分辨率。

4.2.2 为什么使用octomap

RGBD SLAM的目的有两个：估计机器人的轨迹，并建立正确的地图。地图有很多种表达方式，比如特征点地图、网格地图、拓扑地图等等。在《一起做》系列中，我们使用的地图形式主要是点云地图。在程序中，我们根据优化后的位姿，拼接点云，最后构成地图。这种做法很简单，但有一些明显的缺陷：

1. 地图形式不紧凑：点云地图通常规模很大，所以一个pcd文件也会很大。一张640×480的图像，会产生30万个空间点，需要大量的存储空间。即使经过一些滤波之后，pcd文件也是很大的。而且讨厌之处在于，它的“大”并不是必需的。点云地图提供了很多不必要的细节。对于地毯上的褶皱、阴暗处的影子，我们并不特别关心这些东西。把它们放在地图里是浪费空间。
2. 处理重叠的方式不够好：在构建点云时，我们直接按照估计位姿拼在了一起。在位姿存在误差时，会导致地图出现明显的重叠。例如一个电脑屏变成了两个，原本方的边界变成了多边形。对重叠地区的处理方式应该更好一些。
3. 难以用于导航：说起地图的用处，第一就是导航啦！有了地图，就可以指挥机器人从A点到B点运动，但是，我们至少得知道哪些地方可通过，哪些地方不可通过，才能完成导航，光有点是不够的！

4.2.3 八叉树的表达

八叉树(Octree)的概念最早在 1978 年由 Hunter 博士在他的博士毕业论文中提出，是四叉树(Quadtree)在三维空间的推广，主要应用于计算机辅助设计与制造(CAD /CAM)、三维计算机图形学、三维图像处理等领域。用八叉树来表示三维形体，并研究在这种表示下的各种操作及应用是在进入 20 世纪 80 年代后才比较全面地开展起来的。这种方法也可以认为是用三维体素阵列表示形体方法的一种改进。

八叉树结构具有很强的空间分解能力，将空间目标划分为 8 个象限，每一个象限表示八叉树的一个节点。从八叉树根节点开始，如果该节点所在的三维空间实体为空，称其为空节点；如果该节点所在的三维空间实体类型相同(即为均质的)，称其为黑节点；否则，称其为灰节点，并将其沿每一个坐标轴方向分割为 $2 \times 2 \times 2$ 个子节点。这样递归的判断并分割，直到节点为空或该节点所在的三维空间实体为均质的，或达到预先确定的最小尺寸为止。图1 是用八叉树表示简单对象的一个例子，分别用三维栅格结构和树结构描述八叉树。

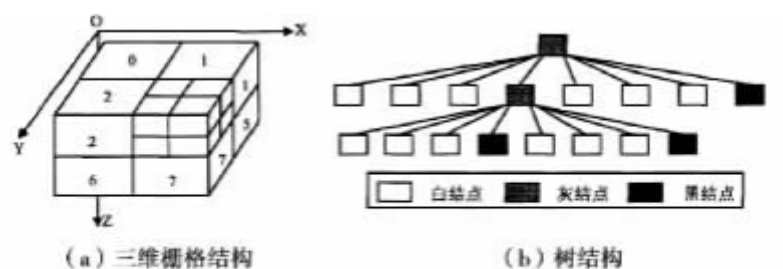


图 1 八叉树表示简单对象

每个节点都有一个数据描述它是否被占据，在最简单的情况下，可以用 0、1 两个数表示，通常用 0 ~ 1 之间的浮点数表示它被占据的概率。0.5 表示未确定，越大则表示被占据的可能性越高，反之亦然。由于它是八叉树，那么一个根节点的 8 个子节点都有一定的概率被占据或不被占据。用树结构的好处在于当某个节点的子节点都占据或“不占据”或“不确定”时，就可以把它减去，这样就可以节省很多空间。

4.2.4 八叉树的更新

在八叉树中，我们用概率来表示一个叶子节点是否被占据而不是直接用 0、1 表达。因为在对环境的观测过程中，由于噪声及物体移动等情况的存在，每个节点是否被占据的情况会随时间的变化而发生改变。根据八叉树的推导原理，假设时间序列为 $t = 1, 2, 3, \dots, T$ ，每个时刻记录的信息为 $i_1, i_2, i_3, \dots, i_T$ 。那么第 n 个子节点的更新信息为：

$$\begin{aligned} P(n|i_{1:T}) \\ = \left[1 + \frac{1 - P(n|i_T)}{P(n|i_T)} \frac{1 - P(n|i_{T-1})}{P(n|i_{T-1})} \frac{P(n)}{1 - P(n)} \right]^{-1} \end{aligned} \quad (1)$$

对上式做一个 logit 变换，把一个概率 p 转换到全实数空间 R 上：

$$\alpha = \text{logit}(p) = \log\left(\frac{p}{1-p}\right) \quad (2)$$

其逆变换为：

$$p = \text{logit}^{-1}(\alpha) = \frac{1}{1 + \exp(-\alpha)} \quad (3)$$

α 称之为 log - odds，用 $L()$ 表示每个叶子节点的 log - odds，则可以得到下面的式子：

$$L(n|i_{1:T}) = L(n|i_T) + L(n|i_{T-1}) \quad (4)$$

通过式(4)可知，每次更新八叉树的信息时，只需要将新的信息加入就可以了，需要注意的是，每个值取值都是有上限和下限的，不然会造成计算溢出，计算八叉树地图的时候只需要把实数转回概率就可以了。通过八叉树

的构造原理可以得到另一个结论是，当需要知道父节点的概率时，可以根据子节点来计算，方法可以是取平均值，或者是取所有子节点的上限或者下限。

4.3 rosbag

4.3.1 rosbag简介

rosbag 既可以指命令行中数据包相关命令，也可以指 c++/python 的 rosbag 库。这里的 rosbag 是指前者。

rosbag 主要用于记录、回放、分析 rostopic 中的数据。它可以将指定 rostopic 中的数据记录到 .bag 后缀的数据包中，便于对其中的数据进行离线分析和处理。

对于 subscribe 某个 topic 的节点来说，它无法区分这个 topic 中的数据到底是实时获取的数据还是从 rosbag 中回放的数据。这就有助于我们基于离线数据快速重现曾经的实际场景，进行可重复、低成本的分析 and 调试。

rosvag对软件包来操作，一个包是ROS用于存储ROS消息数据的文件格式，rosvag命令可以记录、回放和操作包。指令列表如下：

命令	作用
<code>cheak</code>	确定一个包是否可以在当前系统中进行，或者是否可以迁移。
<code>decompress</code>	压缩一个或多个包文件。
<code>filter</code>	解压一个或多个包文件。
<code>fix</code>	在包文件中修复消息，以便在当前系统中播放。
<code>help</code>	获取相关命令指示帮助信息
<code>info</code>	总结一个或多个包文件的内容。
<code>play</code>	以一种时间同步的方式回放一个或多个包文件的内容。
<code>record</code>	用指定主题的内容记录一个包文件。
<code>reindex</code>	重新索引一个或多个包文件。 CSDN @Yangy_Jiaojiao

4.3.2 rosvag具体应用

1. rosvag 录制使用
- rosvag record -a

rosvag record /topic_name

rosvag record -O filename.bag /topic_name

rosvag record -b 4092 /topic_name
- 录制所有话题

录制指定话题

指定生成数据包的名字filename.bag

扩大录制内存限制
2. rosvag info
- rosvag info指令可以显示数据包中的信息：rosvag info filename.bag
3. rosvag 回放使用
- rosvag info name.bag

rosvag play name.bag

rosvag play -r 1.5 name.bag

rosvag play -l name.bag

rosvag play name.bag --topic /topic1
- 查看话题名称、类型、消息数量

回放数据包

1.5倍速回放，按一定频率回放

按一定频率回放，-l选项用来设定循环播放

只播放感兴趣的topic

五、 任务步骤

步骤一 构建真实地图

本任务请参照SLAM建图，构建真实地图。

步骤二 建图

打开桌面自动驾驶汽车桌面管理软件“ROS2.0车”，选择“机器人-LAB”→“深度视觉导航”



激光雷达导航

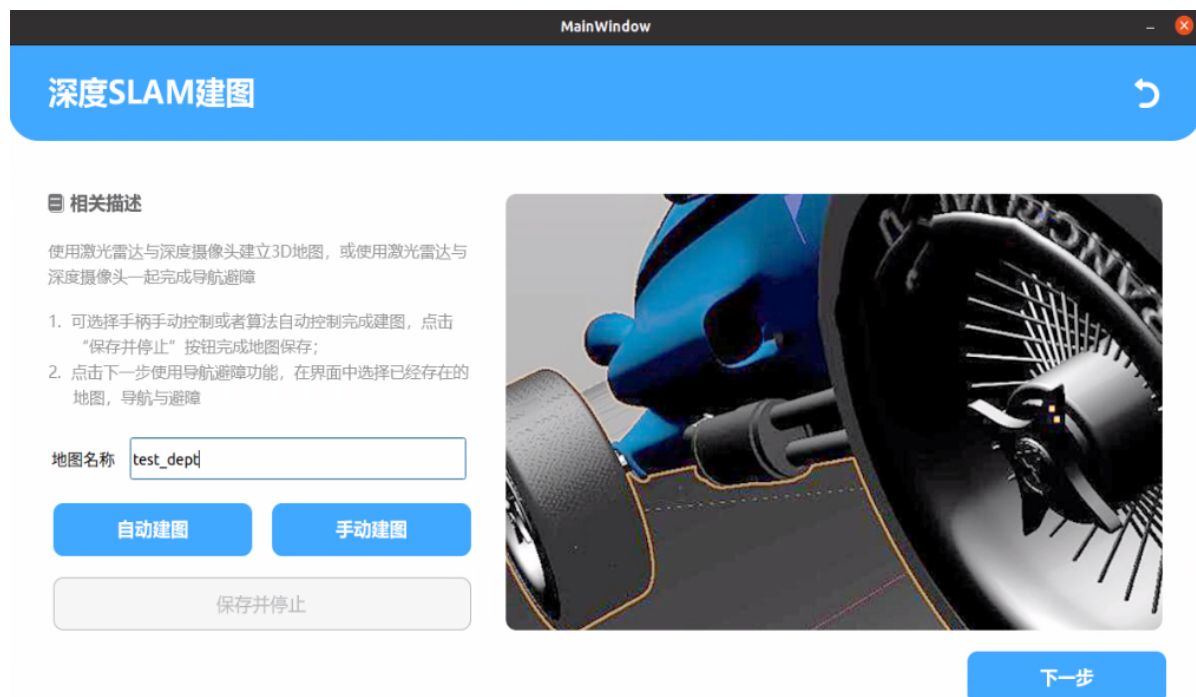


深度视觉导航

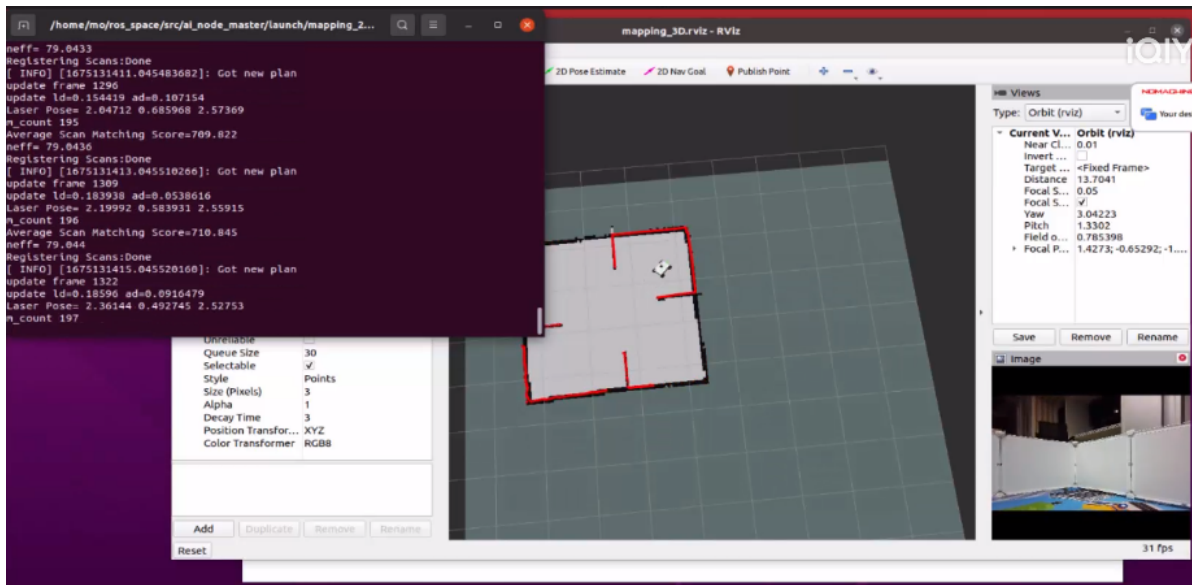


仿真环境

进入“深度SLAM建图”页面：



输入地图名字，点击“自动建图”，打开RVIZ界面，右下角为摄像头视角。



小车在真实地图中按照图上的路线行进：

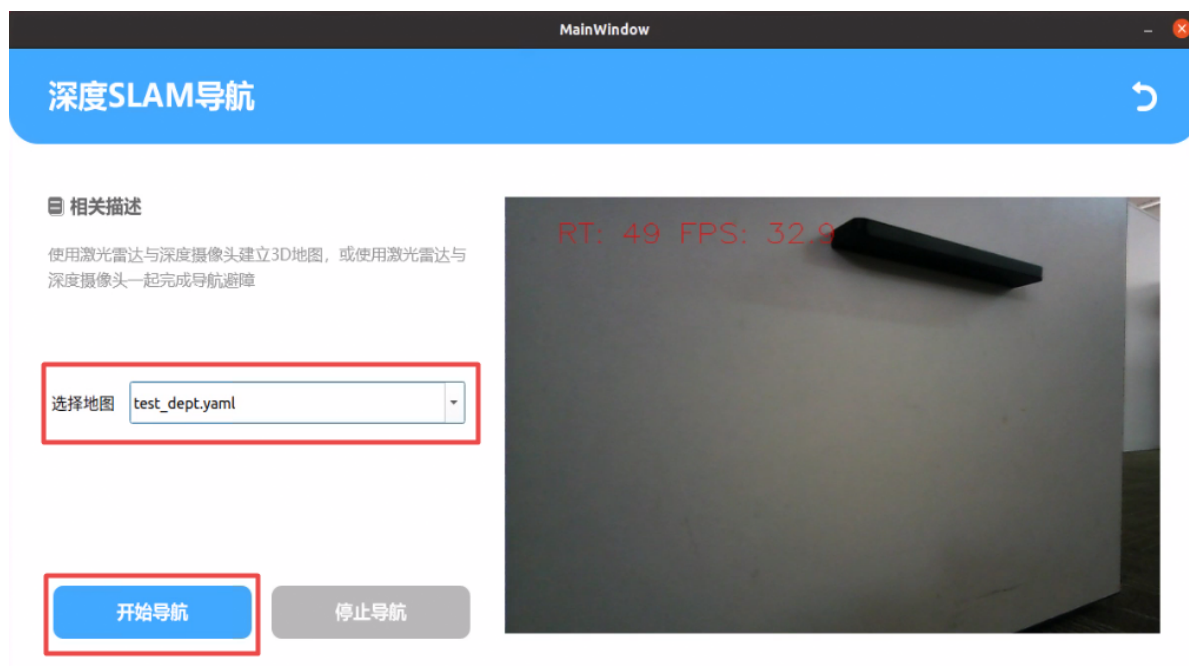


建图结束后，点击“保存并停止”，停止建图并保存创建好的地图：

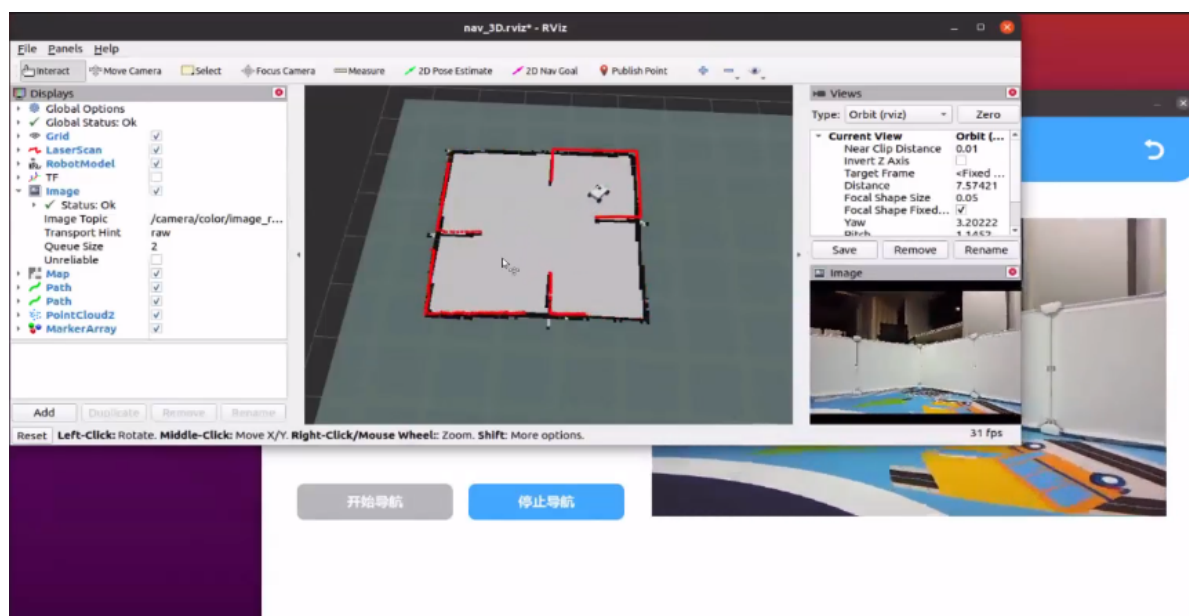


点击“下一步”，进入到导航页面。

步骤三 导航



在“深度SLAM导航”页面，选择创建好的地图，点击“开始导航”，打开RVIZ，可以看到自动驾驶汽车的行驶路线：



在真实地图中，自动驾驶汽车按照RVIZ中的路线行进：



步骤四 代码实现

1 创建文件“/home/mo/ros_space/src/ai_node_master/launch/mapping_3d_explore.launch”，添加如下代码，实现3D自动建图：

```
<launch>
  <!--param参数配置-->
  <param name="node_id_closer"      value="/mapping_3d_explore" />
  <arg name="map"                    default="m_name" />
  <arg name="rotate_speed"           default="0.22" />
  <arg name="rotation_flgae"         default="true" />
  <arg name="nav_sim_flag"           default="false" />
  <arg name="3d_enable"              default="true" />
  <arg name="mapping"                default="true" />
  <node pkg="ai_node_master" type="ros_closer" name="mapping_3d_explore"
output="screen" required="true" />

  <!-- 2D建图节点 -->
  <include file="$(find
navigation_settings)launch/robot_gmapping.launch.xml"/>

  <!-- 激光雷达驱动 -->
  <include file="$(find rplidar_ros)/launch/rplidar.launch" />

  <!-- 底盘与深度摄像头API -->
  <include file="$(find ai_node_master)/cfg/API_cfg.launch" >
</include>

  <!-- 3D建图节点 -->
  <!-- <include file="$(find rtabmap_local)/launch/rgbd_mapping.launch" /> -->
  <include file="$(find octomap_local)/launch/octomap_mapping.launch" />

  <node pkg="ai_node_master" type="3d_map_saver.py" name="map_saver_3d"
output="screen">
    <param name="map_name" value="$(arg map)"/>
  </node>

  <!-- 避障节点 -->
  <include file="$(find navigation_settings)launch/move_base.launch.xml" >
    <arg name="3d_enable"          default="$(arg 3d_enable)" />
    <arg name="mapping"            default="$(arg mapping)" />
  </include>
</launch>
```

```

</include>

<!-- 自动导航节点 -->
<!-- <include file="$(find rrt_exploration)/launch/simple.launch" /> -->
<include file="$(find loop_nav)/launch/loop_nav_3d.launch" >
  <arg name="rotate_speed"    default="$(arg rotate_speed)" />
  <arg name="rotation_flage"   default="$(arg rotation_flage)" />
  <arg name="nav_sim_flag"     default="$(arg nav_sim_flag)" />
</include>

<!-- 深度图像转化雷达数据 -->
<include file="$(find ai_node_master)launch/depth_to_scan.launch">
  <arg name="scan_height"     default="120" />
</include>

```

```

<!-- 可视化界面 -->
<node pkg="rviz" type="rviz" name="rviz" args="-d $(find
urdf_car)/rviz/mapping_3D.rviz" output="screen"/>

</launch>

```

其中以下代码段实现3D建图和地图的保存：

```

<!-- 3D建图节点 -->
<!-- <include file="$(find rtabmap_local)/launch/rgbd_mapping.launch" /> -->
<include file="$(find octomap_local)/launch/octomap_mapping.launch" />

<node pkg="ai_node_master" type="3d_map_saver.py" name="map_saver_3d"
output="screen">
  <param name="map_name" value="$(arg map)"/>
</node>

```

调用“/home/mo/ros_space/src/octomap_local/launch/octomap_mapping.launch”节点，内容如下：

```

<launch>
  <node pkg="octomap_server" type="octomap_color_server_node"
name="octomap_server">
    <param name="resolution" value="0.02" />

    <param name="height_map" value="false" />
    <param name="colored_map" value="true" />

    <param name="pointcloud_max_z" value="0.45" />
    <param name="pointcloud_min_z" value="0.1" />

    <!-- fixed map frame (set to 'map' if SLAM or localization running!) -->
    <param name="frame_id" type="string" value="map" />

    <!-- maximum range to integrate (speedup!) -->
    <param name="sensor_model/max_range" value="1.5" />

    <!-- <param name="latch" value="false" /> -->

```

```

    <!-- data source to integrate (PointCloud2) -->
    <remap from="cloud_in" to="/camera/depth_registered/points" />

  </node>
</launch>

```

此处调用ROS功能：octomap_color_server_node，完成3D建图。

2. 创建文件“/home/mo/ros_space/src/ai_node_master/scripts/3d_map_saver.py”，添加如下代码，实现地图的保存：

```

#!/usr/bin/python3

import signal
import sys,os
import rospy

map_name = ''

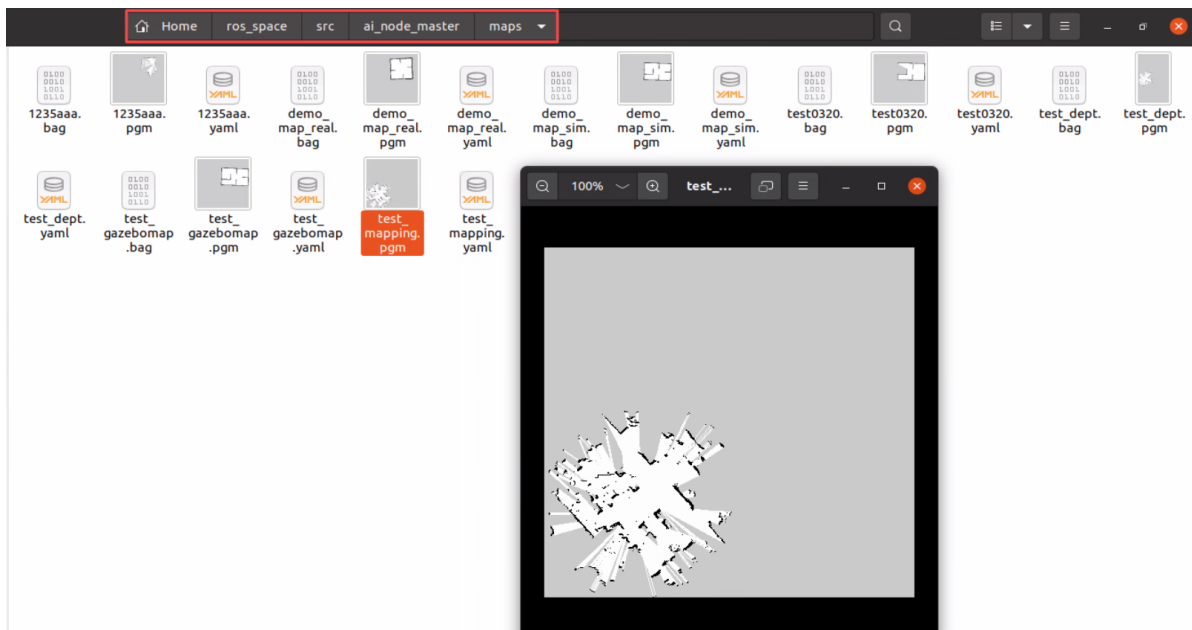
def exit_fn(signum, frame):
    rospy.signal_shutdown("closed!")
    sys.exit()

if __name__ == '__main__':
    rospy.init_node('map_saver_3d',anonymous=True)
    map_name = rospy.get_param('~map_name', 'map')

    signal.signal(signal.SIGINT, exit_fn)
    rate = rospy.Rate(0.25)
    while not rospy.is_shutdown(): # 循环保存地图
        os.system('rosbag record /octomap_point_cloud_centers -O
/home/mo/ros_space/src/ai_node_master/maps/' + str(map_name) + '.bag -l 1')
        rate.sleep()

```

每秒保存一帧，新的地图将覆盖之前的地图，最终的结果保存在“/home/mo/ros_space/src/ai_node_master/maps/”中：



六、任务总结

本实验涉及相机标定、octomap创建3D点云地图和rosbag进行地图的存取。

相机标定是实现准确测量和获取准确位置信息的基础,其主要作用是建立图像像素尺寸与实际尺寸的关系、以及建立图像坐标系与世界坐标系的映射关系。

octomap是依赖于使用八叉树的3D占用网格框架, 而该八叉树使用了概率占用估计。

rosbag主要用于记录、回放、分析 rostopic 中的数据。它可以将指定 rostopic 中的数据记录到 .bag 后缀的数据包中, 便于对其中的数据进行离线分析和处理。